

Assignment 2: DQL, RA, & Triggers

University of Toronto Mississauga

Due: Thursday, Mar 12th, 2026 at 5:00PM ET

Released: Feb 24th, 2026

Overview

For this assignment, we will be working with the *Coruscant Public Transit* (CPT), and perform data analytics over real data, tracking ship routes over the period September 1 - September 7, 2023.

On the CSC343 download site, in `a2.zip`, you will find the following:

- a. `createTables.ddl`, a DDL file for CREATE TABLE statements.
- b. `loadData.ddl`, a DDL file for INSERT TABLE statements.
- c. `A2ERD.png`, the entity-relationship diagram for (a), CPT.
- d. `skeleton.sql`, the skeleton file you must use to put your SQL solutions into.
- e. `skeleton-trigger.sql`, the skeleton file you must use to put the solution to your trigger into.

Please execute scripts `createTables.ddl` and `loadData.ddl` on your PostgreSQL database on the MCS server as you will use this schema for the questions below. Remember, your submission must work on the MCS lab machines! Recall, you can use `\i` to input SQL code and `\o` to output the text from queries!

Part I: Structured Query Language (70 marks)

Write and provide the Structured Query Language (SQL) statements for each of the 10 questions, and their respective subsections using `skeleton.sql`. You must rename this file to `queries.sql`, put your personal information in it, and your answers. This file must be executable.

Execute each of your SQL queries against your CPT database and manually check the result outputted to verify correctness. You may consider writing views or temporary tables for some of these questions; if you do, only add it to your file once! They must be added at the top of the skeleton file in the designated section. You must return any attributes in the order the question below states them to be returned.

- (q1) [3 marks] Find and return the number of seniors (as “Total Seniors”) in the database. A *senior* is defined as being 65-years of age or older (as of the end of 2023). (*Hint*: you may use the `date()` function.)
- (q2) [4 marks] Identify the total number of students who took ship route #1 (i.e., this number is the ‘RouteID’) on September 4th, 2023. Do not specify the column name, use the aggregator.
- (q3) Profitability and revenue from advertising.
 - (a) [3 mark] Identify all your profitable ships. ‘Profitable’ is defined as a ship which has advertising revenue strictly greater than \$10,000. The only information required to be returned from this query is the ships’ unique identifier, age, and manufacturer.

- (b) [3 marks] In addition to (a), above, extend this query by identifying the most profitable ship's pilot(s). Return only their first name, last name, years of service, and the ship's unique identifier.
- (c) [4 marks] Identify the total advertising revenue for each ship's route. The information required to be returned for this query is the ships' route identifier and total advertising revenue (named "Total Revenue"). Results are to be outputted in descending order of Total Revenue.

(q4) Profitability and revenue from fares.

- (a) [4 marks] For each passenger type, find the total fares' revenue. Return the passenger type and its associated total revenue (the column must be named "revenue").
- (b) [2 marks] Extend your query in part (a); only return passenger types and their revenue when the total revenue is greater than \$500. You will return the same column names as in (a).
- (c) [2 marks] Given what you did to solve part (a); return the most profitable passenger type (in terms of total fares' revenue) on September 1, 2023 and their "revenue". You will return the same column names as in (a) and (b).

(q5) Pilots and their violations/infractions.

- (a) [4 marks] Find all pilots who have less than 3 infractions. Return the pilot's unique identification, first name, last name, and age (named "Age"). Note: this query should be dynamic, so use CURRENT_DATE to ensure that when the query is run you get their true age.
- (b) [5 marks] For each pilot, return the total demerit points (named "Total Points") and total fines incurred (named "Total Fine"). Pilots with less than 2 demerit points are irrelevant and can be omitted. Results must be sorted in a manner which places the most offending pilot (by demerit points then highest amount of fines) first on the list.

(q6) Continual growth of the CPT's service.

- (a) [3 marks] Identify what ships are different (i.e., are the only ship made by their manufacturer). Return the ship's unique identification and the name of the manufacturer.
- (b) [4 marks] Determine the most popular ship route on September 7, 2023. 'Popularity' is defined by the largest number of passengers. Return the route's unique identification and the number of passenger trips (as "number of times").
- (c) [4 marks] Find what day contained the largest volume of passenger trips. Return the date and the number of trips taken (as "trips taken").

- (q7) (a) [4 marks] Find all the people who visited any "library" on either September 5, 2023 or September 6, 2023. We want to know their occupation, but we do not want to list duplicates. Therefore, return their occupation and "occurrences", where occurrences reflects the count of the number of times that specific occupation has appeared.
- (b) [3 marks] Extend your query in part (a); return the occupation with the most visits to the library separated by date (i.e., on September 5th, 2023 and on September 6th, 2023).

- (q8) [5 marks] Find the pilots who have worked with CPT for more than 5 years, with a salary greater than \$75,000, and with less than 9 demerit points on their driving record. Return the pilot's first name, last name, and unique identifier.

- (q9) [6 marks] Find all students who attended the “Jedi Knight Basketball” game at the “Jedi Temple” and arrived via a ship on route #4. Return the student’s first name, last name, gender, and phone number. Note: CPT may not have a phone number for all people.
- (q10) [7 marks] Assuming that the ship schedule has not changed since September 2023. Suppose you would like to attend the “Jedi Knight Colloquium” (an event). The colloquium starts at 6:30pm and you would like to arrive at the site between 4:00pm and 6:00pm, which routes can you take? Use September 6, 2023 as a reference date for the ship schedule information. Return the route’s unique identification, the ship’s stop name (i.e., where should you get off the ship), and the scheduled arrival time.
-

Part II: Relational Algebra (30 marks)

This question is a direct extension of the queries for the CPT schema above, please refer back to the sample ERD provided. You will use this ERD to help you solve your questions (not the DDL file provided!).

Write and provide the Relational Algebra (RA) for each of the following 6 questions, which are *similar to* those seen above in the Structured Query Language (SQL) section. The attributes in **bold** are the only columns asked to be returned as output in your answer. You will return them as denoted in the database unless otherwise specified in “quotation marks”.

- (ra1) Find the number of seniors (as “**Total Seniors**”) in the database. A *senior* is defined as being 65-years of age or older (on November 14th, 2023).
- (ra2) Identify all your profitable ships. *Profitable* is defined as a ship which has advertising revenue strictly greater than \$10,000. The only information required from this query is the **ships’ unique identifier**, **age**, and **manufacturer**.
- (ra3) For each pilot, return the **Total Points** and **total fines** incurred (named “**Total Fine**”). Pilots with less than 2 demerit points are irrelevant and can be omitted. Results must be sorted in a manner that places the most offending pilot first on the list (by highest demerit points, if the same, then by the highest amount of fines).
- (ra4) Determine the most popular ship route on September 7, 2023. *Popularity* is defined by the largest number of passengers. Results must provide the **route’s unique identification** and the **number of passenger trips** (as “**number of times**”). Note: you cannot use LIMIT and should use MIN/MAX to get your result.
- (ra5) Find all the people who visited any ‘library’ on either September 5, 2023 or September 6, 2023. We want to know their occupation, but we do not want to list duplicates. Therefore, return their **occupation** and “**occurrences**”, where *occurrences* reflects the count of the number of times that specific occupation has appeared.
- (ra6) Find all students who attended the ‘Jedi Knight Basketball’ game at the ‘Jedi Temple’ and arrived via a ship on route #4. Return the student’s **first name**, **last name**, **gender**, and **phone number**. Note: CPT may not have a phone number for all people.
-

Part III: Triggers (20 marks)

To help with data analytics for CPT, and to make data tracking a bit easier, you are tasked with writing a trigger and function that computes the total earnings based on the ‘Take’ table.

To get started with this question, you will need to create a new schema and then re-run the `createTable.ddl` file. After you have run this file, create a new table called `Earnings` (see `skeleton-trigger.sql` for details):

```
CREATE TABLE IF NOT EXISTS Earnings (  
    Type varchar(20) NOT NULL,  
    TotalEarnings decimal(10,2)  
);
```

The `Earnings` table stores the type of passenger (corresponding `Passenger.Type`) and `TotalEarnings` represents the total amount CPT has earned for that passenger type. You may find question (q4a) from the Standard Query Language section to be particularly useful here!

Your trigger will be named `insert_passenger_fare_trigger` and will execute after an insertion on the `Take` table, while your function will be named `insert_passenger_fare` and will be responsible for updating the `Earnings` table.

Note: if a type does not exist in `Earnings`, then it must be created. Additionally, earnings are to only be computed after an insertion, so if data already exists in the database, you do not add that into this table.

Requirements and Submission

Assignments must be completed individually. Students are forbidden from collaborating with others (i.e., you cannot discuss your solutions/ideas with anyone else, however, you may seek assistance/clarification from the CSC343 Teaching Staff, but not other students, individuals, forums, external services, or external websites). Please read our “Use of Artificial Intelligence (AI)” statement in detail, as all submitted work (i.e., any graded assessments) must be the original work produced by the individual student alone – meaning you are **STRICTLY PROHIBITED FROM USING GENERATIVE AI** (e.g., ChatGPT). Please refer to the syllabus for additional details on assessments and the “Minimum Standards for Submitted Work”.

You are submitting **four files**:

1. **queries.sql** containing your solutions to Part I: SQL.
2. **ra.tex** containing your raw $\text{\LaTeX}$ for Part II: RA.
3. **ra.pdf** containing your compiled $\text{\LaTeX}$ solutions to Part II: RA.
4. **trigger.sql** containing your solutions to Part III: Triggers.

Recall that you **must** include your *personal information* (i.e., full name, student ID number, and UTORid) clearly identifiable in ALL the files you submit. Skeleton files have a designated spot for this, in $\text{\LaTeX}$ files they must go in the preamble (a comment at the top of the file), and in the compiled $\text{\LaTeX}$ PDF it must appear either in the header or on a title page that you create.

Part I: SQL

You are **required** to use the `skeleton.dml` file provided on the ([CSC343 download site](#)). You will rename that file to **queries.sql**, add your SQL queries (i.e., your solutions to Part I: Standard Query Language) to it, and add your *personal information* where indicated to do so. You must ensure that your SQL statements are syntactically correct and that they are executable on the MCS PostgreSQL server. Non-executable files or queries (or those producing no output) will receive a grade of 0. Failure to use or follow the structure of the skeleton file (this includes modifying the structure of the skeleton file) will result in a grade of 0. Do not leave your solutions commented out; if so, a penalty will be applied.

Part II: RA

In **ra.tex** and **ra.pdf**, you should include your answers for the relational algebra questions. Note: no hand-written solution will be accepted. In **ra.tex**, you must include your *personal information* at the top of the file in the preamble. In **ra.pdf**, it must appear either in the header of the file or on a front title page that you create.

Part III: Triggers

You are **required** to use the `skeleton_trigger.sql` file provided on the ([CSC343 download site](#)). You will rename that file to **trigger.sql** and add the solution to Part III: Triggers; you must add your SQL function and trigger (i.e., your solutions) and *personal information* where indicated to do so. Please ensure that your SQL code is syntactically correct and that they are executable on the MCS PostgreSQL server. Non-executable functions or triggers (or those producing no result/output) will receive a grade of 0. Do not leave your code commented out; if so, a penalty will be applied.

Submission on MarkUs Platform

All files are to be submitted using the MarkUs platform (<https://mcsmarks.utm.utoronto.ca/markus26s/courses/8/>). You may submit as many times as you like, in fact you are encouraged to submit many times! Please ensure your answers are typed and submissions are legible/understandable.

Please see the course syllabus for all policies relating to this assignment (e.g., late submissions, marking, grace tokens, etc.)